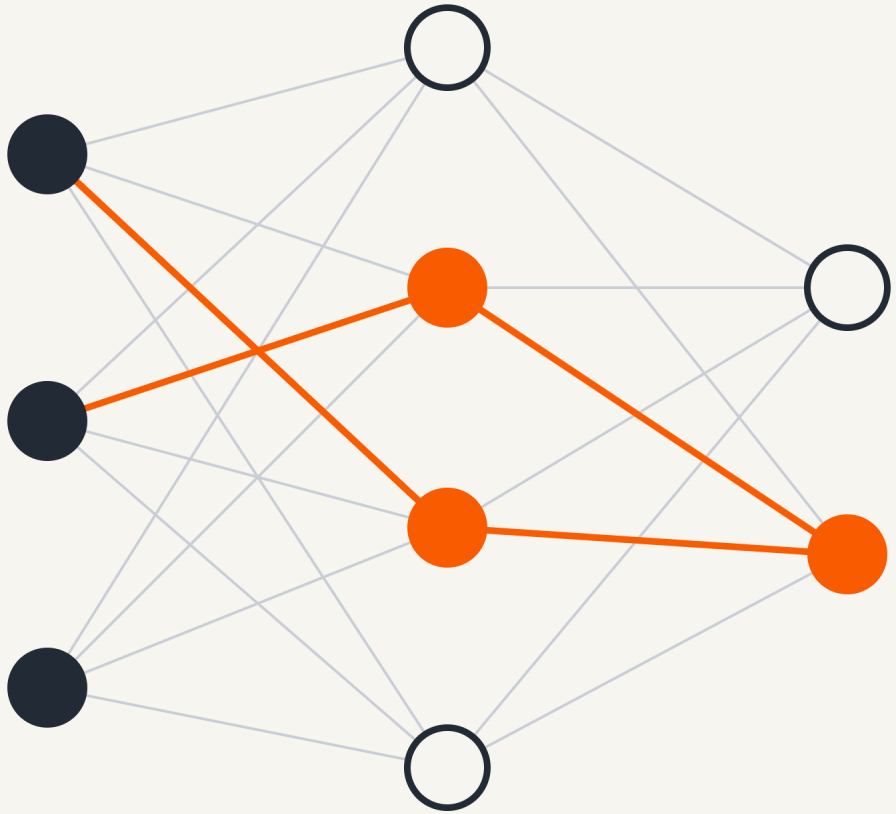
DocGaid

KNOW-HOW SHARE · AI ENGINEERING

Neural networks, demystified



What they are, what they aren’t, and where they actually fit in AI systems

Yanek Hayrapetyan 29/10/2026

A QUICK QUIZ

One of these is a photo.

The other was painted by a small neural network that has never seen an image. It only knows how to answer one question: “what colour is the point (x, y)?”

Which is which? We’ll build it live.



A



B

THE WHOLE TALK IN ONE SENTENCE

A neural network is a program whose logic is found by search, not written by hand.

You choose the shape of the program and supply examples. An optimizer fills in millions of numbers until the examples pass.

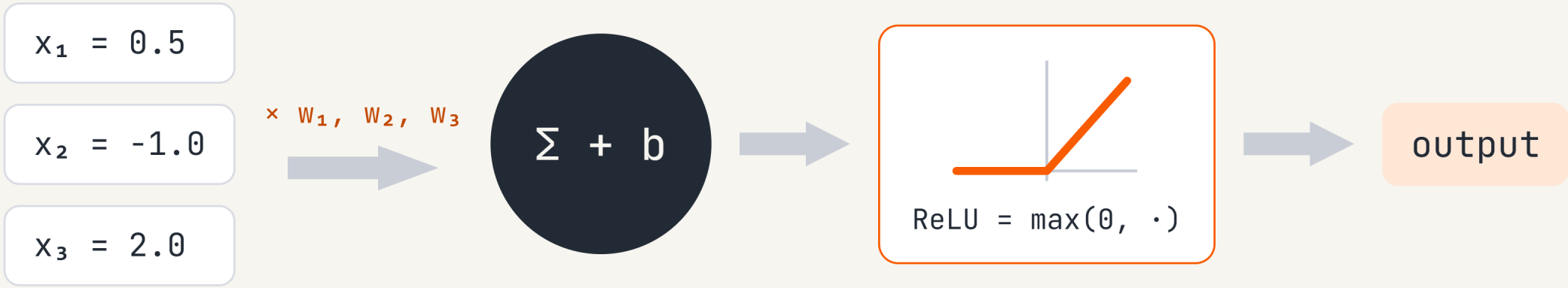
TRANSLATING TO SOFTWARE TERMS

You already know most of this

Software you write	A neural network
Function signature	Architecture: input shape → output shape
Function body	Weights: millions of numbers instead of lines of code
Writing the code	Training: an optimizer nudges the numbers
Test suite	Loss function + examples: a score for “how wrong”
Specification	The data: bad data means a bad spec
Hardcoding test answers	Overfitting: passes training data, fails in production

THE BUILDING BLOCK

A neuron: a weighted sum and a squiggle

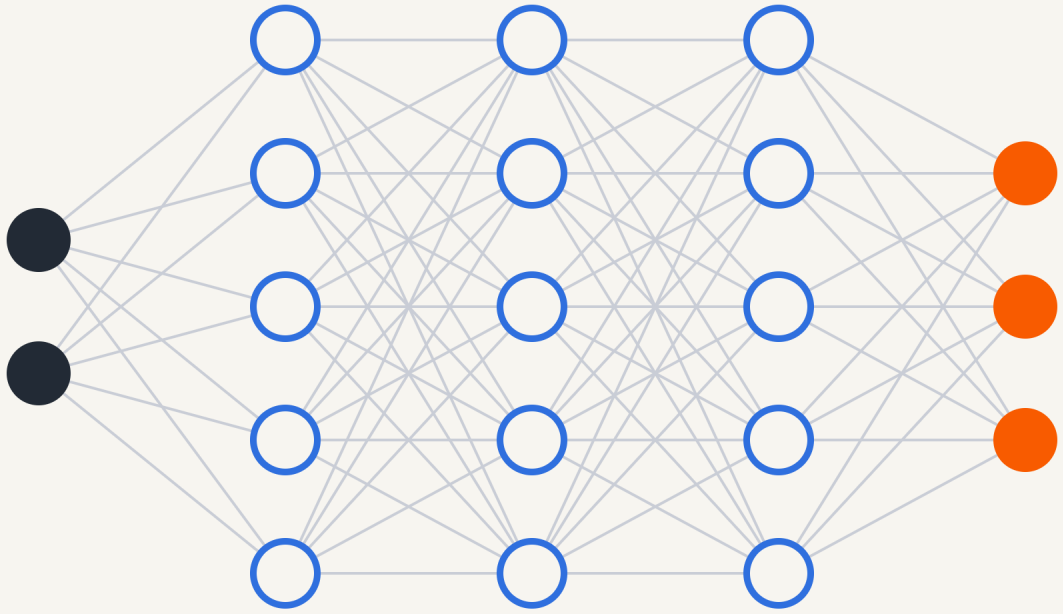


$$y = \mathrm{ReLU}(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

The weights **w** and the bias **b** are the **only** things that are ever learned.

FROM NEURONS TO A NETWORK

Stack neurons in layers: an MLP



input hidden layers output

Multi-Layer Perceptron

The “hello world” architecture. Every neuron feeds every neuron in the next layer.

Hidden layers

Intermediate representations the network invents for itself. Nobody labels them.

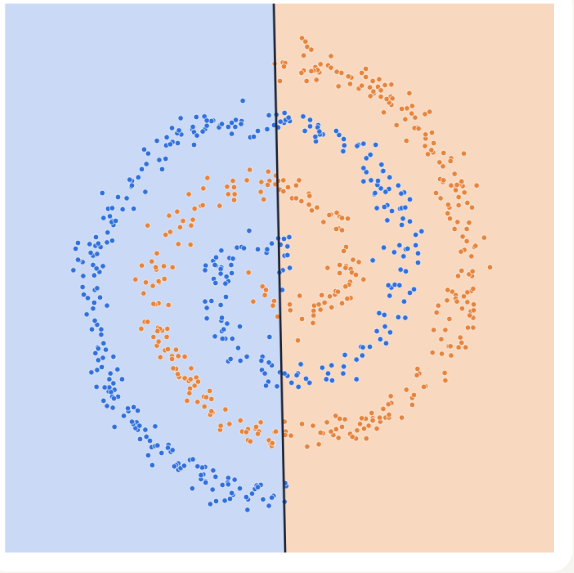
Everything else builds on it

CNNs, transformers and LLMs are MLPs plus extra structure.

WHY THE SQUIGGLE MATTERS

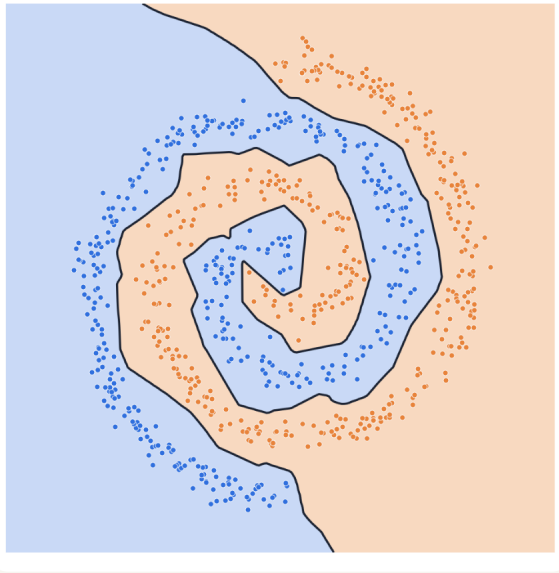
Without it, depth is useless

$$W_3 \cdot W_2 \cdot W_1 \cdot x = W \cdot x$$



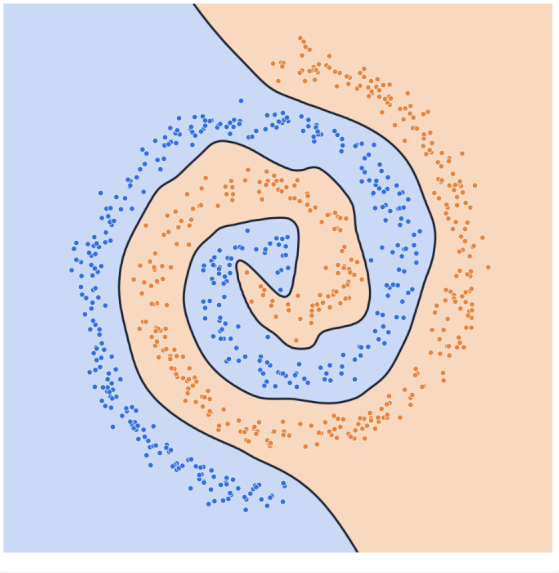
No activation · 68%

3 layers, still one straight line.



ReLU · 100%

Space folded from straight pieces.



Tanh · 100%

Smooth curves.

HOW IT LEARNS

Training: one loop, on repeat

1

Forward

Run the inputs through. Make a guess.

2

Loss

One number: how wrong was the guess?

3

Backward

Which weights are to blame, and how much?

4

Update

Nudge every weight a tiny bit to reduce the loss.

```
for step in
range(N):
    guess = model(x)                    # 1 forward
    loss = loss_fn(guess, y)            # 2 loss
    loss.backward()                     # 3 backward: chain rule, automated
    optimizer.step()                    # 4 update
```

THE INTUITION BEHIND STEP 4

Gradient descent: walk downhill



Horizontal axis: a weight’s value. Vertical axis: the loss.

Gradient = slope

For every weight: “if I increase you a little, does the loss go up or down?”

Learning rate = step size

Too small: painfully slow. Too large: you overshoot and bounce around.

Millions of dimensions

One axis per weight. Nobody sees the landscape; you only feel the local slope.

LIVE DEMO

An image is just a function

$$f(x, y) \rightarrow (r, g, b)$$

INPUT

A pixel's position, scaled to 0…1

OUTPUT

Its colour: three numbers, 0…1

DATASET

One image. Every pixel is one example.

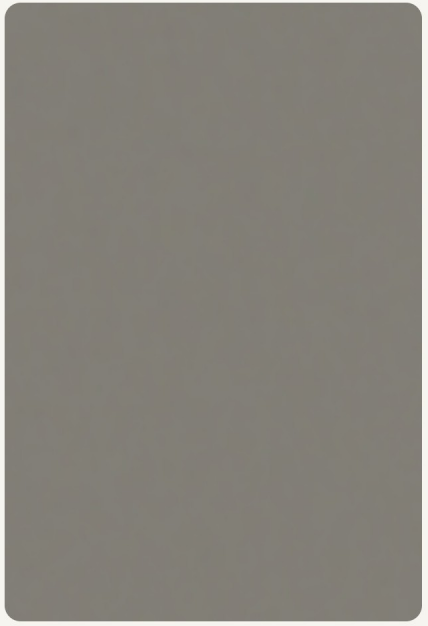
LOSS

Squared error between predicted and true colour

Here the network is **supposed** to memorise. That lets us see the whole learned function at once: it's the picture.

DEMO BACKUP · WATCHING IT LEARN

From grey blob to cat in 2,000 steps



step 0



step 25



step 100



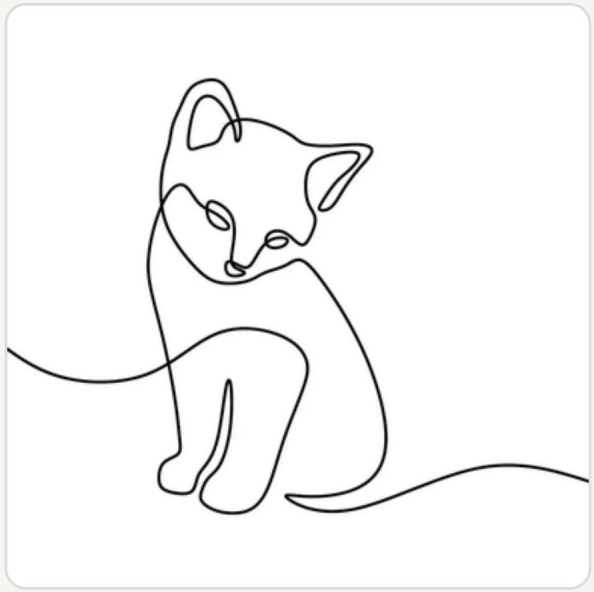
step 300



step 2,000 (= image A)

EXPERIMENT · SPECTRAL BIAS

Same network, different input

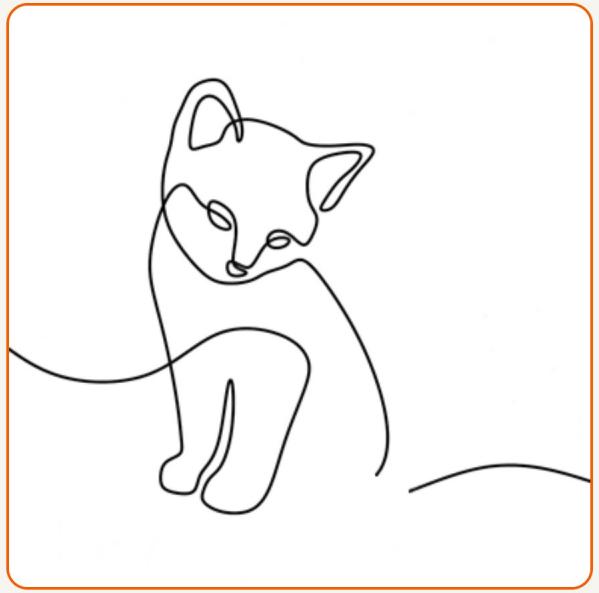


Target

Thin lines = sharp, high-frequency detail

Raw (x, y): blank

It learned one thing: “the paper is white”.



Fourier features: sharp

Feed sin/cos of (x, y) at many frequencies.

Lesson: networks learn smooth patterns first. How you **represent the input** can matter more than model size.

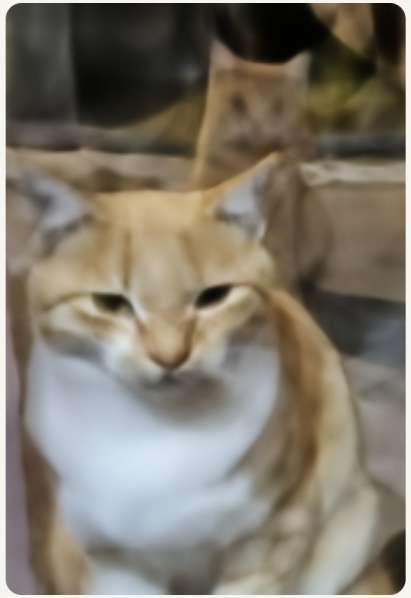
EXPERIMENT · HIDE HALF THE PIXELS

Memorising is not learning



What it sees

50% of pixels hidden: our test set



σ = 1 · blurry

train 28.3 dB
test 27.0 dB



σ = 10 · best

train 48.3 dB
test **28.7 dB**



σ = 100 · overfit

train 44.8 dB
test **10.0 dB**

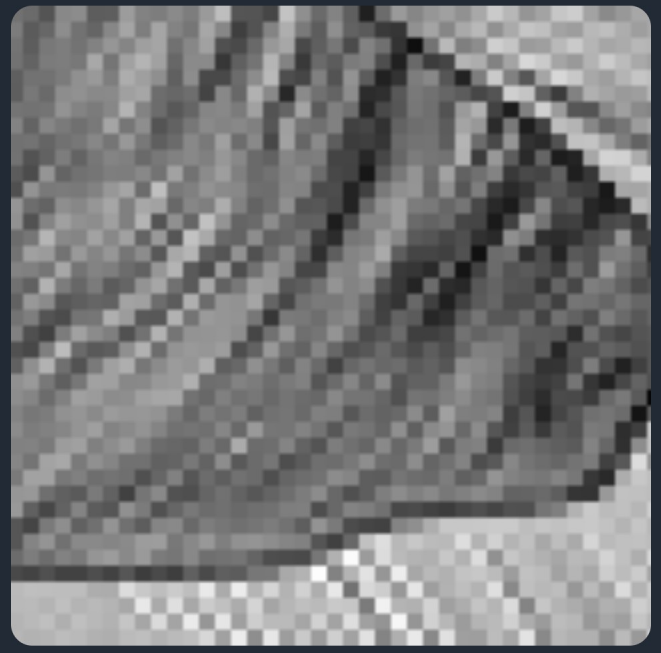
PSNR measures closeness to the real image. Higher is better; +10 dB = 10× less error.

BONUS · IT'S A FUNCTION, NOT A GRID

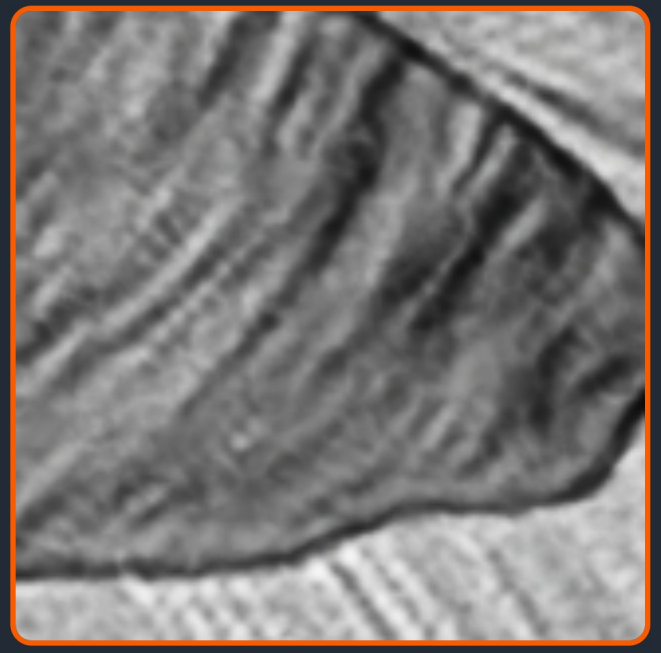
Ask for colours between the pixels

Trained on a small image, rendered at 4× the resolution. The network fills in values it was never shown.

The same idea in 3D, (x, y, z, view) → (colour, density), is **NeRF**: 3D scenes from photos.



training pixels, zoomed 4×



network rendered at 4×

MISCONCEPTIONS · 1 OF 2

What neural networks are not

MYTH

“It works like the human brain.”

REALITY

Loosely inspired by neurons in the 1940s–50s. In practice it’s linear algebra plus calculus. Brains don’t learn by backprop.

MYTH

“AI means neural networks.”

REALITY

Rules, search, and classic ML are AI too. On tabular data, gradient-boosted trees often still win.

MYTH

“It can learn anything: it’s a theorem!”

REALITY

The universal approximation theorem says a solution *exists*. Not that training finds it, nor that it generalizes.

MISCONCEPTIONS · 2 OF 2

What working with them is really like

MYTH

“You need millions of examples.”

REALITY

Not if you start from a pretrained model. Fine-tuning often works with hundreds or a few thousand examples.

MYTH

“Training is the hard part.”

REALITY

The training loop is four lines. Data quality, evaluation and monitoring are where the time goes.

MYTH

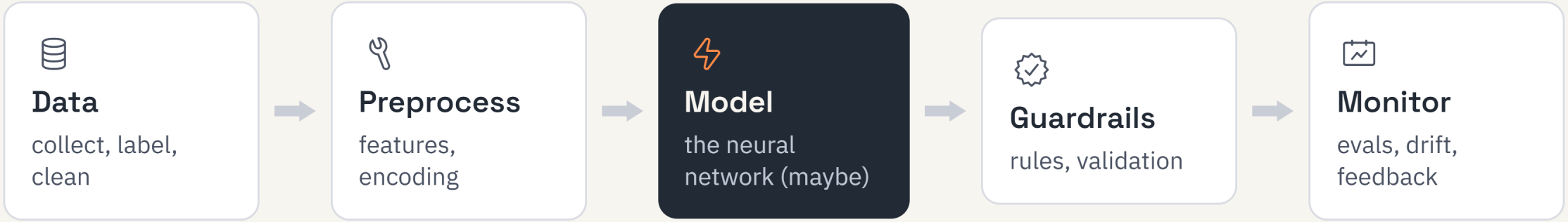
“It’s a black box: you can’t test it.”

REALITY

Opaque, not unknowable. Held-out test sets and evals test behaviour; interpretability tools probe the inside.

WHERE THEY FIT

The model is one box in the pipeline



Most of the engineering lives around the model, not inside it.

Google's "Hidden Technical Debt in ML Systems" (2015) made the same point: the ML code is a small fraction of a real-world ML system.

IS A NEURAL NETWORK MANDATORY? NO.

Pick the simplest tool that works

If your problem looks like...	A good first choice
Clear, stable rules (validation, routing)	Plain code. Testable, explainable, free.
Tables: churn, pricing, fraud scores	Gradient-boosted trees (XGBoost, LightGBM)
Little data, must explain every decision	Linear models or decision trees
Images, audio, free text	Neural networks, almost always pretrained
Search and similarity	Embeddings (from a network) + a vector index
Language: extract, summarise, generate	An LLM (a huge network) + engineering around it

Neural networks shine on **unstructured** data, where nobody can write the rules by hand.

LLM PRODUCTS

The network is the engine, not the car

Context & prompts
What the model sees, and how it's asked

Retrieval (RAG)
Your documents, found by embeddings

Tools & APIs
Real actions and exact computations

Guardrails
Validate outputs, schemas, policies

The LLM
one (very big) neural network

Evals & monitoring
Test suites for behaviour, in CI and production

Five of these six boxes are ordinary software engineering. That's where you come in.

IF YOU REMEMBER FIVE THINGS

Takeaways

1. A neural network is a program whose logic is **found by search**, not written.
2. The **nonlinearity** is what makes depth useful. Without it, it’s one matrix.
3. Training is always the same loop: **forward, loss, backward, update**.
4. **Representation** and **held-out evaluation** matter more than model size.
5. The network is **one box** in a pipeline, and often not the one you need.

Questions?

The notebook is yours to play with:
neural_networks_demo.ipynb

GO DEEPER

3Blue1Brown: Neural Networks
The best visual intuition, video series

Karpathy: Neural Networks, Zero to Hero
Build backprop and a GPT from scratch, in code

TensorFlow Playground
playground.tensorflow.org: the spirals, in your browser

Tancik et al., 2020: Fourier Features
The paper behind the blurry-vs-sharp demo

Sculley et al., 2015: Hidden Technical Debt
Why the model is the small box